

Programación III — Guía Complementaria

Cómo Determinar el Parámetro 'k' en las Ecuaciones de Recurrencia

Cátedra: Programación III

Tema: Análisis de Trabajo No Recursivo

1. Concepto Fundamental de 'k'

En el análisis de algoritmos recursivos (métodos de **Sustracción** y **División**), la función de costo se expresa formalmente como:

- **Sustracción:** $T(n) = a \cdot T(n - b) + p(n)$

- **División:** $T(n) = a \cdot T(n / b) + f(n)$

El término $p(n)$ o $f(n)$ representa el **trabajo no recursivo** que realiza la función para una entrada de tamaño n . El parámetro k es el **grado del polinomio que describe ese trabajo extra**.

2. Metodología Paso a Paso para Identificar 'k'

1. **Ignora el Caso Base:** El caso base procesa tamaños constantes ($n = 1$ o $n = 0$). Lo que determina k es el trabajo realizado en el caso recursivo.
2. **Aísla el Código No Recursivo:** "Tapa" visualmente o ignora las llamadas a la función misma. Analiza las asignaciones, condicionales, operaciones aritméticas y bucles restantes.
3. **Calcula la Complejidad Externa:**
 - Si solo hay operaciones elementales (asignaciones, ``return``, ``if``) $\Rightarrow$ Costo $O(1) = O(n^0) \Rightarrow k = 0$.
 - Si hay un bucle dependiente de $n \Rightarrow$ Costo $O(n) = O(n^1) \Rightarrow k = 1$.
 - Si hay dos bucles anidados dependientes de $n \Rightarrow$ Costo $O(n^2) \Rightarrow k = 2$.

3. Ejemplos Prácticos con Código Java

Caso 1: $k = 0$ (Operaciones Elementales Constantes)

```
int factorial(int n) {  
    if (n == 1) return 1;  
    else return n * factorial(n - 1); // Multiplicación y retorno: O(1)  
}
```

Análisis: La instrucción `n * ...` y el `return` son operaciones aritméticas y de control simples. No dependen de un bucle. Su costo es $O(1) \Rightarrow k = 0$.

Caso 2: $k = 1$ (Un Bucle Lineal Dependiente de n)

```
public void procesarYDividir(int[] v, int inicio, int fin) {
    int n = fin - inicio + 1;
    if (n <= 1) return;

    // --- TRABAJO NO RECURSIVO ---
    for (int i = inicio; i <= fin; i++) { // Bucle de 0 a n
        System.out.print(v[i] + " ");
    }

    int medio = (inicio + fin) / 2;
    procesarYDividir(v, inicio, medio);
    procesarYDividir(v, medio + 1, fin);
}
```

Análisis: El bucle `for` recorre los n elementos del subarreglo realizando impresiones. Su complejidad es $O(n) \Rightarrow k = 1$. (Ejemplo clásico del paso de combinación en *MergeSort*).

Caso 3: $k = 2$ (Bucles Anidados Cuadráticos)

```
public void matrizRecursiva(int n) {
    if (n <= 1) return;

    // --- TRABAJO NO RECURSIVO ---
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.println(i + ", " + j);
        }
    }

    matrizRecursiva(n / 2);
}
```

Análisis: Fuera de la llamada recursiva existen dos bucles anidados que ejecutan $n \times n = n^2$ impresiones. Su complejidad es $O(n^2) \Rightarrow k = 2$.

4. Cuadro de Referencia Rápida para Estudio

Código fuera de las Llamadas Recursivas	Complejidad del Trabajo Extra	Polinomio Representativo	Valor de 'k'
Asignaciones, comparaciones, <code>`return`</code> , operaciones aritméticas	Constante $O(1)$	$c \cdot n^0$	$k = 0$
Un bucle <code>`for`</code> o <code>`while`</code> que va desde 0 hasta n	Lineal $O(n)$	$c \cdot n^1$	$k = 1$
Dos bucles anidados que recorren la entrada hasta n	Cuadrática $O(n^2)$	$c \cdot n^2$	$k = 2$